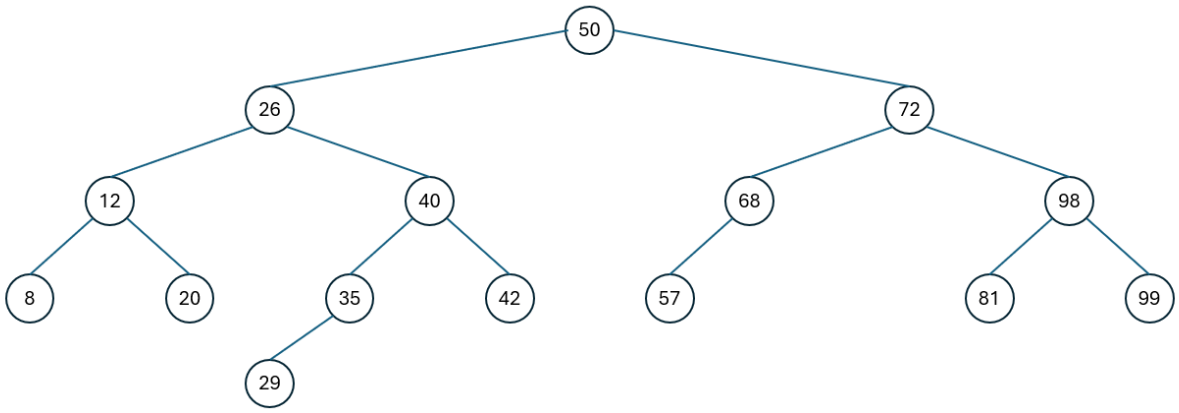
Jeffrey Smith
Dr. Z

1. Draw a Binary Tree

Draw a binary search tree created from these keys:

first														last
50	72	68	26	98	12	8	40	57	99	20	42	35	81	29

Figure 1. Keys to construct a binary tree from



2. Level Order Traversal of a Binary Tree

Perform a level-order traversal of this tree.

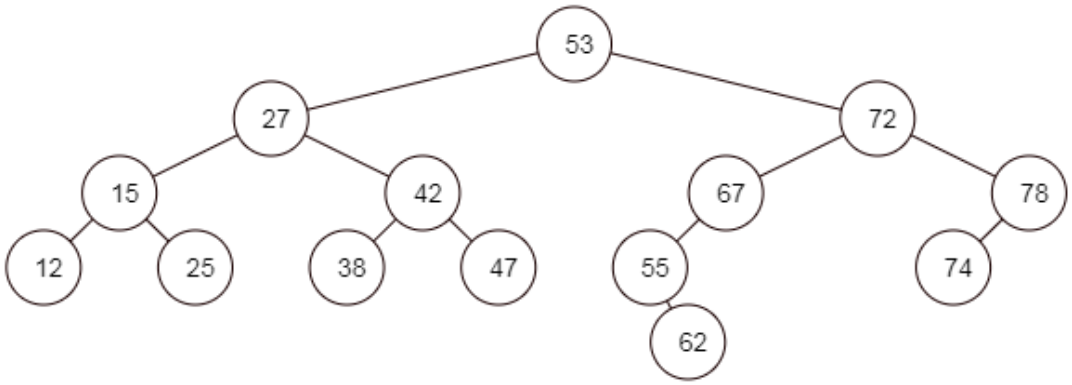


Figure 2. Binary tree for level-order traversal

first													last
53	27	72	15	42	67	78	12	25	38	47	55	74	62

3. Translate Level Order Traversal into Words

To make it more interesting, a traversal can be decoded into words. The decoder is in *Figure 3*. Each key in the tree maps to a different letter in the decoder. For example, when you see key 53 in the tree, it matches to a 'N' in the decoder.

Key	53	27	15	12	25	42	38	47	72	67	55	62	78	74
Letter	N	e	e	i	v	r	e	-	v	-	u	!	g	p

Figure 3: Decoder for level-order keys

In your design notebook:

- Traverse the tree in *level-order*, writing down the numbers as you go.
- Decode the message associated with the level order traversal.

Translation: Never-give-up!

Step 1: Problem Statement

This assignment provides the opportunity to implement a binary search tree and to implement a level-order traversal of the tree. The tree will be filled with parrot objects and when a level order traversal is performed the parrots will "sing" a song.

The first step is to fill a binary search tree with parrot objects. Each parrot has an ID number, name, and one word that is part of the song. The parrots are placed in the binary tree using their IDs as keys. After the tree is filled, traverse the tree in level-order, and have each parrot sing its word in the song as you visit the node containing the parrot. Finally, list the names of the parrots who are at the leaves of the tree (going from left to right).

Step 2: What I know

Creating the classes, reading from files, creating the objects, displaying information, all pretty straight forward.

Step 2: What I don't know

Populating the tree is something new. Also, applying the overridden compareTo() function has been a real challenge for me thus far. Hoping it goes more smoothly this time through. Lastly, at least before I try to do it, reading the tree both at level order and reading just the leaves will be a first and a bit of a challenge.

Jeffrey Smith
Dr. Z

Step 3: Pseudo-code (main only)

```
class main {

        //create file object to read from
        file myFile = new file("parrots.txt");

        // create scanner objects to read file
        scanner myScan = new scanner(myFile);

        // create binary tree
        binaryTree myTree = new binaryTree();

        // read file to create parrot objects
        while (myScan.hasNext) {

                int id = myScan.nextInt;
                String name = myScan.Next;
                String song = myScan.Next;

                parrot myParrot = new parrot(id, name, song);

                //add parrot to tree
                binaryTree.Add(myParrot);

        }

        // traverse tree and print word by calling levelOrder();
        binaryTree.levelOrder();

        // call visit leaves to visit leaves from left to right and print parrot name in each leaf
        binaryTree.visitLeaves();

} // end main
```

Step 4: What I Learned

Well, good news is that the Overridden compareTo() method was actually a breeze this time through. Additionally, populating the tree was pretty straightforward. Reading level order from the tree was more of a challenge to understand how the code was functioning than to write the code, and, lastly, reading from just the leaves was the same. The code seemed simple enough, but I had a hard time understanding the order in which operations were happening which made it a real challenge. I kept trying to step through the code as if it were a regular loop rather than fully grasping the order in which the recursive operations were happening. It was quite a challenge and had to do some research to get the code just right, but was able to get it all working. Loved the secret message!